

Travail pratique 1

IFT 2035 — Été 2026

Remise via Studium au plus tard le 31 mai 2026 à 23h59.

Le TP se réalise en équipe de deux personnes.

1 Mini-Haskell

Le TP 1 consiste à compléter l'implantation d'un interpréteur pour un petit langage fonctionnel appelé **mini-Haskell**. Comme son grand frère, mini-Haskell est fonctionnel pur, typé statiquement et à portée lexicale ; la syntaxe est à base de S-expressions (comme LISP / Scheme). La grammaire complète, les règles de typage et les règles d'évaluation sont présentées à l'annexe A.

2 Votre travail

2.1 Fichiers fournis et structure générale

Quatre fichiers vous sont fournis :

`Parseur.hs` Parseur de S-expressions (utilise `Parsec`). À ne pas modifier.
`Main.hs` Programme principal (`repl` et `unittests`). À ne pas modifier.
`Eval.hs` Datatypes et fonctions de l'interpréteur. **C'est le seul fichier à modifier.**
`unittests.txt` Tests unitaires. Vous pouvez ajouter des tests si vous le souhaitez.

L'interpréteur transforme du texte source en valeur selon le pipeline :

$$\text{String} \xrightarrow{\text{p0neSexp}} \text{Sexp} \xrightarrow{\text{semp2Exp}} \text{Exp} \xrightarrow{\text{typeCheck}} \text{Exp} \xrightarrow{\text{eval}} \text{Value}$$

Les fonctions `semp2Exp`, `typeCheck` et `eval` se trouvent dans `Eval.hs`. `eval` n'est appelée que si `typeCheck` réussit ; elle ne doit donc jamais planter sur une expression bien typée.

2.2 Ce qui est déjà fourni dans le squelette

Le fichier `Eval.hs` contient déjà :

- Les **datatypes** `Type`, `Exp` et `Value` ainsi que les environnements initiaux `env0` (opérateurs `+`, `-`, `*`) et `tenv0`.
- `semp2Exp` pour les entiers, les variables, les `lambda` à plusieurs paramètres (désués en `lambdas` imbriqués), les applications à plusieurs arguments (désués en applications imbriquées), et le `let`. Les cas `data` et `case` sont à compléter.
- `typeCheck` et `eval` pour les entiers et les variables.

Il vous donc reste à implanter `sexp2Exp` pour les constructions `data` et `case`, ainsi que `typeCheck` et `eval` pour les lambdas (`ELam`), les applications (`EApp`), les types algébriques (`EData`) et le filtrage par motifs (`ECase`).

2.3 Étape 1 — Lambda, application et let

Implantez `typeCheck` et `eval` pour les expressions `lambda`, les applications de fonction et le `let` mutuellement récursif.

Représentation dans l'ASA.

- Un `lambda` est `ELam sym type corps` : paramètre `sym` de type `type` et corps `corps`. Dans l'ASA, chaque `lambda` n'a toujours qu'un seul paramètre (le sucre syntaxique multi-paramètres est désucré par `sexp2Exp`).
- Une application est `EApp fonction argument` (un argument à la fois).
- Les types sont `TInt` (entier), `TArrow t1 t2` (fonction $t_1 \rightarrow t_2$) et `TData nom` (type algébrique).
- Les valeurs sont `VInt n`, `VLam param corps env_capture` (fermeture) et `VPrim f` (opérateur primitif).

Notation des types fonction.

Dans la syntaxe de mini-Haskell, un type fonction s'écrit sous forme de liste : `(Int Int)` désigne $\text{Int} \rightarrow \text{Int}$, `(Int Int Int)` désigne $\text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$, `(Bool Bool)` désigne $\text{Bool} \rightarrow \text{Bool}$, etc. Les règles de typage précises sont à l'annexe A.2.

Let mutuellement récursif.

Toutes les variables déclarées dans un même `let` sont visibles dans toutes les définitions et dans le corps. Pour `typeCheck`, étendez l'environnement avec les types déclarés avant de vérifier chaque expression de définition. Pour `eval`, la paresse de Haskell permet d'exprimer la récursion mutuelle naturellement.

Exemples attendus.

<code>((lambda ((x Int)) x) 4)</code>	<code>-- resultat : 4</code>
<code>(+ 1 2)</code>	<code>-- resultat : 3</code>
<code>((lambda ((x Int)) (lambda ((y Int)) (+ x y)))</code> <code>6) 8)</code>	<code>-- resultat :</code>
<code>14</code>	
<code>(let ((x Int 5)) (* x x))</code>	<code>-- resultat : 25</code>

2.4 Étape 2 — Types algébriques et filtrage par motifs

2.4.1 sexp2Exp pour data et case

Complétez `sexp2Exp` pour les constructions `data` et `case`. La structure syntaxique exacte est donnée à l'annexe A.1 (figure 1).

Important : une expression `data` imbriquée à l'intérieur d'une autre expression doit être rejetée par `sexp2Exp`.

2.4.2 typeCheck pour EData et ECase

Les règles précises sont à l'annexe A.2. En résumé :

- **EData** : vérifier l'unicité des noms de types et constructeurs, rejeter la redéfinition de `Int`, calculer le type de chaque constructeur, puis vérifier le corps dans l'environnement augmenté.
- **ECase** : vérifier que l'expression scrutinée est de type `TData`, que chaque branche utilise un constructeur valide avec le bon nombre de variables, et que toutes les branches retournent le même type.

2.4.3 eval pour EData et ECase

- **EData** : ajouter les constructeurs dans l'environnement. Un constructeur sans argument est une valeur `VData` immédiate ; un constructeur avec k arguments est une chaîne de fonctions qui accumule les arguments avant de construire la valeur finale. Évaluer ensuite le corps dans l'environnement augmenté.
- **ECase** : évaluer l'expression scrutinée, trouver la première branche dont le constructeur correspond, puis évaluer l'expression de cette branche dans l'environnement étendu avec les variables du motif liées aux arguments.

Exemples attendus.

```
(data ((Bool True False))
  (let ((x Bool True)) x))                                -- resultat : True

(data ((ListInt Nil (Cons Int ListInt)))
  (case (Cons 4 Nil)
    ((Nil 0)
      ((Cons x xs) x))))                                -- resultat : 4

(data ((Bool True False))
  (let ((neg (Bool Bool)
            (lambda ((x Bool))
              (case x ((True False) (False True))))))
    (neg True)))                                         -- resultat :
False
```

2.5 Tests et utilisation

Tout au long du processus de développement, vous êtes encouragés à tester votre implémentation de manière interactive. Chargez `Main.hs` dans `GHCi` pour tester interactivement :

```
$ ghci
Prelude> :l Main.hs
```

```
*Main> repl
MiniHaskell> (+ 1 2)
3 :: Int
MiniHaskell> :q
Leaving Mini Haskell
```

Vous pouvez aussi exécuter les tests unitaires fournis avec la commande suivante dans GHCi une fois `Main.hs` chargé :

```
*Main> unittests "unittests.txt"
```

Chaque test est une paire `(e1 e2)` : succès si `e1` et `e2` s'évaluent à la même valeur. Un test `(e1 Erreur)` (avec majuscule) réussit si `e1` déclenche une erreur dans `sexp2Exp` ou `typeCheck`. **Les tests font partie intégrante de la spécification du langage.**

3 Rapport

Remettez un rapport de **3 pages** (page titre non comprise), en français, couvrant :

1. Une description de votre implantation des cas `ELam`, `EApp`, `EData` et `ECase` dans `typeCheck` et `eval`, ainsi que de `sexp2Exp` pour `data` et `case`.
2. Les difficultés rencontrées. Si des tests échouent, expliquez pourquoi et proposez une piste de solution.
3. Votre expérience générale avec Haskell et avec ce TP.

4 Évaluation

La note finale du TP 1 sera basée sur les critères suivants :

- **Tests unitaires** : proportion de tests réussis. Une partie des tests est fournie, d'autres tests seront ajoutés pour la correction.
- **Qualité du code** : lisibilité, style Haskell idiomatique, absence de code mort.
- **Rapport** : clarté et complétude.

Les notes possibles, en pourcentage, pour le TP sont : 0%, 30%, 50%, 60%, 70%, 80%, 90% ou 100%.

- 100% : tous les tests réussis, code de bonne qualité, rapport clair et complet.
- 90% : une ou deux petites erreurs mineures, qui ne cachent pas un problème de compréhension.
- 80% : quelques erreurs, dont au moins une qui révèle une incompréhension partielle du sujet.
- 60% - 70% : plusieurs erreurs, mais une compréhension générale.
- 0%, 30%, 50% : des erreurs majeures qui révèlent un manque de compréhension du sujet, ou une implantation très incomplète.

Une ou des questions sur le TP pourront être posées lors de l'examen final.

$global$	$::= e$	une expression
	$(data\ (t_1 \dots t_n)\ e)$	définition de types algébriques
e	$::= n$	entier signé
	x	variable
	$(e_1\ e_2 \dots e_n)$	application de fonction ($n \geq 2$)
	$(lambda\ ((x_1\ \tau_1)\ \dots\ (x_n\ \tau_n))\ e)$	définition de fonction ($n \geq 1$)
	$(let\ (d_1 \dots d_n)\ e)$	déclarations locales ($n \geq 1$)
	$(case\ e\ (m_1 \dots m_n))$	filtrage par motifs ($n \geq 1$)
τ	$::= Int$	type des entiers
	$(\tau_1\ \tau_2 \dots \tau_n)$	type fonction : $\tau_1 \rightarrow \tau_2 \rightarrow \dots \rightarrow \tau_n$
	dt	type déclaré par data
t	$::= (dt\ c_1 \dots c_n)$	un type et ses constructeurs
c	$::= x$	constructeur sans argument
	$(x\ \tau_1 \dots \tau_n)$	constructeur avec arguments
d	$::= (x\ \tau\ e)$	déclaration dans un let
m	$::= (cm\ e)$	branche du case
cm	$::= x$	constructeur sans argument
	$(x\ x_1 \dots x_n)$	constructeur avec arguments et variables liées

FIGURE 1 – Grammaire EBNF de mini-Haskell

A Spécification de mini-Haskell

A.1 Syntaxe

La syntaxe de mini-Haskell est basée sur les *S-expressions* : une notation préfixe parenthésée similaire à LISP / Scheme. La grammaire EBNF est présentée à la figure 1 et des exemples à la figure 2.

Remarques importantes.

- **Espaces significatifs.** $(+1\ 1)$ est l'application de la variable $+1$ à l'argument 1, et non $(+ 1\ 1)$.
- **Type fonction.** La liste $(\tau_1\ \tau_2)$ désigne $\tau_1 \rightarrow \tau_2$. Exemples : $(Int\ Int) = Int \rightarrow Int$, $(Int\ Int\ Int) = Int \rightarrow Int \rightarrow Int$, $(Bool\ Bool) = Bool \rightarrow Bool$.
- **data au niveau global uniquement.** Toute expression **data** imbriquée est une erreur de syntaxe.
- **Sucre syntaxique.** $(lambda\ ((x\ Int)\ (y\ Int))\ e) \equiv (lambda\ ((x\ Int))\ (lambda\ ((y\ Int))\ e))$. $(f\ e1\ e2) \equiv ((f\ e1)\ e2)$.

```

-- Application des operateurs integres (+ - *)
(+ 1 2)
(* 3 4)

-- Lambda a plusieurs parametres (sucre syntaxique)
(lambda ((x Int) (y Int)) (+ x y))

-- Let (mutuellement recursif)
(let ((x Int 5)
      (y Int 8))
    (+ x y))

-- Datatype et case
(data ((Bool True False))
    (let ((x Bool True)) x))

(data ((ListInt Nil (Cons Int ListInt)))
    (case (Cons 4 Nil)
      ((Nil 0)
       ((Cons x xs) x))))

```

FIGURE 2 – Exemples d’expressions mini-Haskell valides

A.2 Règles de typage

L’environnement de types Γ associe un type à chaque variable et constructeur. L’environnement initial Γ_0 contient $+$, $-$, $*$ de type $\text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$.

Entier et variable

$$\frac{}{\Gamma \vdash n : \text{Int}} \text{NUM} \qquad \frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau} \text{VAR}$$

Lambda et application

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash (\text{lambda } ((x \tau_1)) e) : \tau_1 \rightarrow \tau_2} \text{LAM} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash (e_1 e_2) : \tau_2} \text{APP}$$

Let

Le `let` est mutuellement récursif : toutes les variables déclarées sont visibles dans toutes les définitions et dans le corps. Les noms de variables d’un même `let` doivent être distincts.

$$\frac{\begin{array}{c} \Gamma' = \Gamma, x_1 : \tau_1, \dots, x_n : \tau_n \\ \forall i. \Gamma' \vdash e_i : \tau_i \quad \Gamma' \vdash e : \tau \end{array}}{\Gamma \vdash (\text{let } ((x_1 \tau_1 e_1) \dots (x_n \tau_n e_n)) e) : \tau} \text{LET}$$

Data

Tous les noms de types et de constructeurs introduits doivent être uniques et **Int** ne peut pas être redéfini. Pour un type dt , un constructeur sans paramètre a le type dt , et un constructeur avec k paramètres de types $\tau_1, \dots, \tau_k$ a le type $\tau_1 \rightarrow \dots \rightarrow \tau_k \rightarrow dt$.

$$\frac{\begin{array}{c} \text{noms uniques, Int non redéfini} \\ \forall i, j. c_{ij} = (x_{ij} \tau_{ij_1} \dots \tau_{ij_k}) \Rightarrow \tau_{ij} = \tau_{ij_1} \rightarrow \dots \rightarrow \tau_{ij_k} \rightarrow dt_i \\ \Gamma, c_{11} : \tau_{11}, \dots, c_{nm} : \tau_{nm} \vdash e : \tau \end{array}}{\Gamma \vdash (\text{data } (t_1 \dots t_n) e) : \tau} \text{DATA}$$

Case

$$\frac{\begin{array}{c} \Gamma \vdash e : dt \\ \forall i. \Gamma(x_i) = \tau_{i_1} \rightarrow \dots \rightarrow \tau_{i_m} \rightarrow dt \\ \forall i. \Gamma, x_{i_1} : \tau_{i_1}, \dots, x_{i_m} : \tau_{i_m} \vdash e_i : \tau \end{array}}{\Gamma \vdash (\text{case } e ((cm_1 e_1) \dots (cm_n e_n))) : \tau} \text{CASE}$$

où cm_i est soit x_i (constructeur sans argument) soit $(x_i x_{i_1} \dots x_{i_m})$ (avec arguments liés).

A.3 Règles d'évaluation

Toute expression se réduit à une *valeur* : entier, fermeture, ou objet construit par un constructeur de **data**.

- **Entier** : s'évalue à lui-même.
- **Lambda** : s'évalue en une fermeture capturant l'environnement courant.
- **Application** : évalue la fonction, puis l'argument, puis applique (en étendant l'environnement de capture avec le paramètre).
- **Let** : évalue le corps dans l'environnement augmenté des définitions. Les définitions sont *mutuellement récursives* : l'ordre de déclaration n'a pas d'importance.
- **Data** : introduit les constructeurs dans l'environnement d'évaluation, puis évalue le corps.
- **Case** : évalue l'expression scrutinée, puis entre dans la *première* branche dont le constructeur correspond.

En cas d'ambiguïté sur la sémantique, les tests unitaires font foi.